Programming for Computational Linguistics 2025/2026

Exercise Set 10 (2026-01-15)
Time and Memory Issues

Once you have completed all exercises, send an email to the progclgrader@ims.uni-stuttgart.de with the title "submit set10", and with your python files as attachments. You should receive a response with feedback and a grade.

Exercises are due on **2026-02-03 23:59**, but we encourage you to do them during the lab session.

Exercise 1. Create a module `maxima.py` with a function `find_maxima(lst)`. The function should return a new list with all the elements from `lst` that are bigger than **all** the elements which precede them. For example, a result of `find_maxima([1, 3, 2, 7, 4])` should be `[1, 3, 7]` because `1` is first (so there are no preceding elements), `3` is bigger than `1`, and `7` is bigger than all `[1, 3, 2]`. Other examples:

- `find_maxima([])` returns `[]`
- `find_maxima([1])` returns `[1]`
- `find_maxima([4,1,3])` returns `[4]`
- `find_maxima([2,5,3,1,6,6,7])` returns `[2, 5, 6, 7]`

Make sure that your solution has complexity $O(n)$ (so it is **linear**). Running `find_maxima(range(100000))` should finish in less than **10 seconds**.

Exercise 2. We will call two words "opposite" if one is equal to the other read backwards. For example, words "deer" and "reed" are opposite. Create a file `opposites.py` with a function `find_opposites(lst)`. The function should accept as a parameter a list of words `lst` and return a list of **all** pairs of opposite words from `lst`. For example, for a list:

```
[ "according", "deer", "net", "ten", "reed", "refer",
  "raw", "war", "addition", "frequency", "platform" ]
```

your function should return: `[('deer', 'reed'), ('net', 'ten'), ('raw', 'war')]`
The result should:

- contain pairs (tuples) of opposite words (the order of pairs and words within one pair is not relevant).
- contain every pair of words only once – notice that there is `('deer', 'reed')` but no `('reed', 'deer')`
- not contain palindromes (words which are opposite to themselves)

Other examples:

- `find_opposites([])` returns `[]`
- `find_opposites(["abba"])` returns `[]`
- `find_opposites(["deer", "reed"])` returns `[ ("deer", "reed") ]`

Function `find_opposites` should work efficiently and be able to handle very long lists of words. Before submitting your solution test it locally on your computer. Download from ILIAS file `words.txt` and script `check_fast_opposites.py`. Put them in the same directory as your solution and run `check_fast_opposites.py`. The code should finish within **less than 10 seconds** and find 857 pairs of opposite words.

Exercise 3. A sparse vector is a vector in which most of the elements are zero. For example `[0, 0, 0, 0, 0, 1, 0, 0, 0, 2, 0, 1, 0, 0, 0, 0, 10, 0, 0, 0]` can represent a 20-dimensional sparse vector with non-zero elements on positions 5, 9, 11, and 16.

For big dimensions the representation above wastes a lot of memory, as it keeps information about all the zeros and their positions. In a file `sparse.py` write your own representation of a sparse vector `SparseVector`.

- The constructor of `SparseVector` should accept `length` (dimensionality) of the vector as an argument:

```
v1 = SparseVector(10)
v2 = SparseVector(5)
v3 = SparseVector(100000000000)
```

  The `SparseVector` should use memory efficiently, so running the code above should not cause `MemoryError`.

- With an appropriate method make sure that running `len` on your object returns the dimension of the vector:

```
print("Len of v1 =", len(v1))
print("Len of v2 =", len(v2))
print("Len of v3 =", len(v3))
```

```
Len of v1 = 10
Len of v2 = 5
Len of v3 = 100000000000
```

  The method should have time complexity $O(1)$.

- In your class write methods `set_pos(pos, el)`, which will set the element on the given position and `get_pos(pos)` which will retrieve it:

```
print("v1 at pos 4 =", v1.get_pos(4))
v1.set_pos(4, 2)
print("v1 at pos 4 =", v1.get_pos(4))

print("v2 at pos 0 =", v2.get_pos(0))
v2.set_pos(0, 7)
print("v2 at pos 0 =", v2.get_pos(0))
```

```
v1 at pos 4 = 0
v1 at pos 4 = 2
v2 at pos 0 = 0
v2 at pos 0 = 7
```

  Both of the methods should have constant time complexity.

- With an appropriate method make sure that running `str` on your object returns the string representation of it:

```
print("v1 is", str(v1))
print("v2 is", str(v2))
```

```
v1 is SparseVector[0,0,0,0,2,0,0,0,0,0]
v2 is SparseVector[7,0,0,0,0]
```

  This method has to be $O(n)$ because it generates all the positions. Be careful not to run it on very big n.

- Write a method `concatenate(v)` which will take as an argument another `SparseVector` and return **a new** `SparseVector` which will be a concatenation of the two vectors:

```
v1_v2 = v1.concatenate(v2)
print("v1_v2 is", str(v1_v2))
```

```
v1_v2 is SparseVector[0,0,0,0,2,0,0,0,0,0,7,0,0,0,0]
```

The time complexity of this method should not depend on n but on the number of non-zero elements. Running code below should end after less than **10s**:

```
v3.set_pos(100, 43)

v4 = SparseVector(1000000000)
v4.set_pos(106, 23)

v3_v4 = v3.concatenate(v4)
print("v3_v4 at pos 100000000106 =",
      v3_v4.get_pos(100000000106))
```

```
v3_v4 at pos 100000000106 = 23
```